CHOOSEWISE.EDUCATION

ADVANCED GUIDE · CLAUDE CODE

Build your own AI OS

A practical blueprint for educators who want one working system instead of twenty open chat windows, built with Claude Code and without being a developer.

Johan Lindström

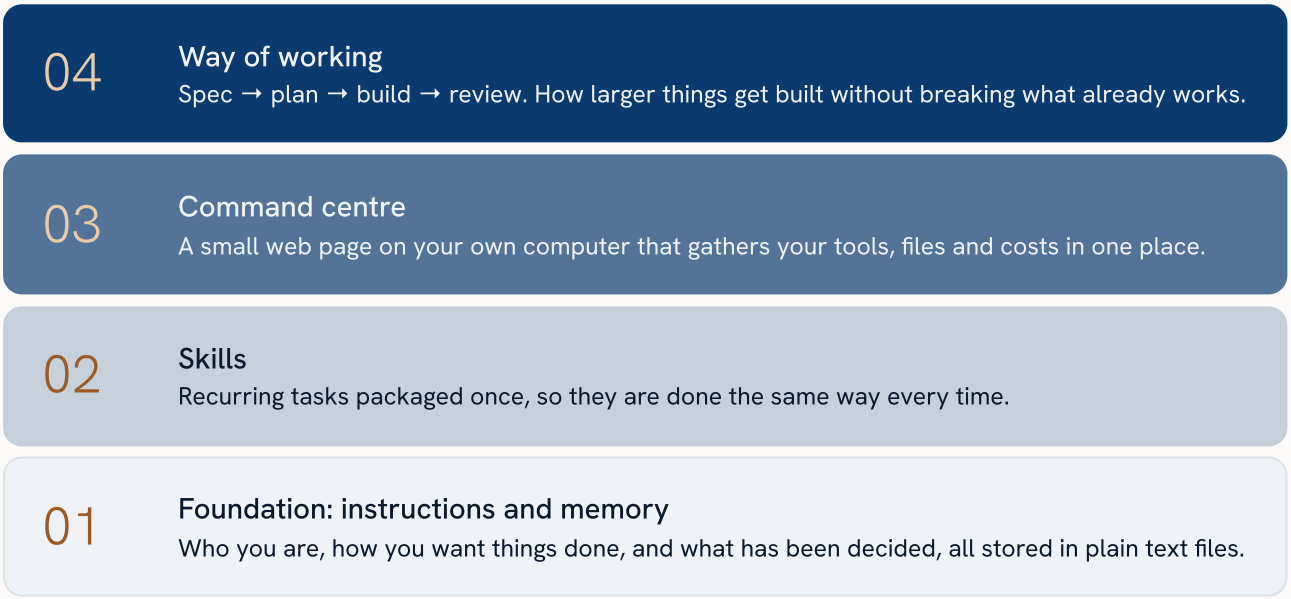
October 2026

INTRODUCTION

From chat windows to a system that remembers

Most educators use AI the same way: a new chat, a long explanation of who they are and what they need, a decent answer, and then the whole context disappears when the window closes. The next day the explanation starts again. That is not a skills problem. It is an architecture problem: nothing you teach the AI is stored anywhere you control.

An **AI OS** (a personal AI operating system) solves that by moving the context out of the chat and into files on your own computer. Your instructions, your memory, your recurring ways of working and your tools live in folders that Claude Code reads every time it starts. The system grows with every task instead of starting from zero.



Build from the bottom up. Each layer only works if the one beneath it is in place.

> JOHAN'S NOTE
>
> I built mine in the autumn of 2026, when I went independent and needed one place that held several brands, several projects and the decisions behind them. The order in this guide is not the order I built it in. It is the order I would build it in if I started again: foundation first, because every layer above it reads from it.

BEFORE YOU START

Twenty minutes in the Terminal is enough

This is an advanced project, but you do not need to be a developer. Claude Code runs in the **Terminal**, a window where you type instead of click. You type a handful of commands yourself. Everything else you write in plain English, and Claude types the commands for you.

What you need

- A Mac or a Windows computer that you control (not a locked-down school device).
- A paid Claude plan (Pro, Max, Team or Enterprise). The free plan does not include Claude Code.

Install and start

Follow Anthropic's setup page, code.claude.com/docs/en/setup. On a Mac, open the Terminal (`⌘ Space` , type *Terminal*) and paste:

```
curl -fsSL https://claude.ai/install.sh | bash
```

Then create a folder for your system, step into it and start Claude Code:

```
mkdir ~/ai-os
cd ~/ai-os
claude
```

The first time, you log in with your Claude account. After that you are in an ordinary conversation, except that Claude can now read and write files in this folder. (Anthropic also offers a desktop app that runs Claude Code without the Terminal. It is a gentle start, but the Terminal is where the whole system is visible.)

Three commands to recognise

`cd` moves you into a folder, and Claude Code always works in the folder you start it from. `ls` lists what is in the folder. `claude` starts Claude Code, and `/exit` leaves it.

> WATCH OUT
>
> Read what Claude Code asks to run before you approve it. If you do not understand a command, ask: *"What does this do, and can it be undone?"* That habit keeps you safe.

01

PART 1 · FOUNDATION

Instructions: the file Claude reads every time

Everything you would otherwise explain at the start of every chat belongs in one file. Write it once, and keep sharpening it.

Claude Code reads a file called `CLAUDE.md` before every session. A `CLAUDE.md` in your home folder applies everywhere, and one inside a project folder applies only there. This file is your operating manual for Claude, and it is the single highest-value thing in the whole system.

What belongs in it

- Who you are, and your experience
- Your audiences and the tone for each
- Your brands or hats, and how to tell them apart
- Hard rules, marked MUST and NEVER
- A short checklist to run before every reply

What stays out

- Long background texts (link to a file instead)
- Project status (that goes in memory)
- Passwords, keys, student data. Ever.
- Rules you would not actually enforce

PROMPT · WRITE YOUR CLAUDE.MD

Interview me, one question at a time, so that you can write my personal CLAUDE.md in my home folder. Ask about my role and experience, my audiences, the tone I want for each, the brands or roles I work under, the mistakes you must never make, and the tools I already use. When you have enough, show me a draft before you save it. Write every hard rule with a short reason, so you can apply it to cases the rule does not mention.

JOHAN'S NOTE

My most useful rule is the one that says "if you are unsure which brand this is for, ask before you produce anything". One question costs ten seconds. Content in the wrong voice costs an afternoon. Write the rules that protect you from your own most expensive mistakes.

PART 1 · FOUNDATION

Memory: what Claude should know next time

Instructions describe how you want things done. Memory holds what has *happened*: decisions, project status, lessons learned and where things live. Claude Code has a built-in memory, but you get the most from it when you decide its shape yourself.

One fact per file, plus an index

Keep memory as a folder of short text files, each holding one fact, and one index file that lists them all with a one-line description. The index is loaded at the start of every session; the individual files are read when they are relevant. Four kinds of memory cover almost everything:

- **About you**: role, preferences, how you like to work.
- **Feedback**: corrections you have given, with the reason, so they are not repeated.
- **Projects**: status, decisions and deadlines, with absolute dates.
- **References**: where things are: folders, accounts, external documents.

Reference documents

Some knowledge is too big for memory: a brand guide, your voice and writing style, the positioning of a product. Give each its own document, store it in the folder of the brand it belongs to, and point to it from `CLAUDE.md` with an instruction such as *"read this before every task for this brand"*.

PROMPT · SET UP MEMORY

Set up a memory for me: one folder, one file per fact with a short header (name, description, type), and an index file with one line per memory. From now on, save anything worth remembering between sessions without me asking, and update an existing memory instead of creating a duplicate. Start by saving what you learned about me in this session.

> WATCH OUT
>
> Decide early where memory lives. Mine ended up spread across twelve project folders before I gathered it in one. And treat memory as a record of what was true when it was written: ask Claude to check a remembered file or setting before acting on it.

02

PART 2 · SKILLS

Package what you do more than twice

A skill is a way of working written down once, so it is carried out the same way every time, by you, by Claude, or by both.

A **skill** is a folder containing a file called `SKILL.md` : a short description of when it should be used, followed by the instructions. Personal skills live in `~/.claude/skills/` . Claude Code sees the descriptions of all your skills and uses the right one when a task matches. You can also call one directly by typing a slash and its name, for example `/lesson-plan` .

A task deserves a skill when you have explained the same procedure three times, when the result must look the same every time (a lesson plan, a newsletter, a feedback format), or when it has steps that are easy to forget, such as a quality check before delivery.

PROMPT · TURN A TASK INTO A SKILL

We have just done [task] together. Turn the way we did it into a personal skill. Write a description that says exactly when it should be used, using words I would actually say. Include the steps, the quality checks and the mistakes we corrected along the way. Show me the skill before you save it, then test it on a new example.

Skills from other people

There are thousands of shared skills and plugins. The one most worth installing early is **superpowers**, a plugin that gives Claude Code a disciplined way of working: brainstorming before building, written plans, tests and review. Part 4 is built on it. Install it from inside Claude Code:

```
/plugin install superpowers@claude-plugins-official
```

> WATCH OUT
>
> A skill can include scripts that run on your computer. Before installing someone else's, ask Claude to review it for anything that sends data elsewhere or runs code. And curate: with a hundred vaguely described skills, the wrong one gets picked. Group them, and delete what you no longer use.

03

PART 3 · COMMAND CENTRE

One page that gathers everything

When the system holds more than you can keep in your head, give it a face: a small web page that runs only on your own computer.

The command centre is a local web page that you open in your browser at an address like `localhost:4000`. No one else can reach it, and nothing is uploaded anywhere. Claude Code builds it and maintains it. You describe what you want to see. You never need to write the code yourself.

Six principles that keep it maintainable

1. **Local only.** The server listens only to your own computer, never to the network.
2. **Panels, not one big app.** Each tab is its own small app with its own folder, so one broken panel never takes the others down.
3. **As few dependencies as possible.** Plain HTML, CSS and JavaScript, run by Node. Fewer moving parts means fewer things that break after an update.
4. **Starts by itself.** It starts when you log in and restarts if it crashes. You should never have to remember to start it.
5. **Keys in one separate file.** API keys live in one file that is never shared or put under version control.
6. **Every action shows its cost.** Anything that calls a paid service logs what it cost.

PROMPT · BUILD THE SHELL

I want a personal command centre: a local web page at localhost that only my own computer can reach. Build the shell first: a simple server, a start page with tabs, and a rule that every tab is a separate panel in its own folder. Use as few dependencies as possible. Make it start automatically when I log in. Before you write any code, describe the plan in plain language and wait for my approval.

PART 3 · COMMAND CENTRE

Build one panel at a time

Start with the panel that saves you the most friction this week, not the most impressive one. Each panel below has a starter prompt. Run them one at a time, and use each panel for a few days before you build the next.

PANEL · TODAY

Add a Today panel as the start page: buttons that open the programs and folders I use every day, today's open tasks from a simple list I can edit, and a short status line for each of my projects, read from my memory files.

PANEL · SKILLS

Add a Skills panel that reads all my installed skills and commands from disk and shows them in groups with their descriptions. Flag skills whose description is too vague to trigger reliably. Store my grouping in a file in the project.

PANEL · MEDIA LIBRARY

Add a Media panel: a searchable library of the images and videos I generate, with the prompt, the model, the date and the cost saved next to each file. Let me generate new images from the panel, and show the cost before anything is sent.

PANEL · COSTS

Add a Costs panel that shows what I have spent today, this week and this month, per service, using the cost log the other panels write to.

> **JOHAN'S NOTE**
>
> The lesson that cost me most time: a server that starts automatically keeps running the code it was started with. After every change, ask Claude to restart it before you test anything. Otherwise you will spend an evening debugging a bug that was fixed hours ago.

04

PART 4 · WAY OF WORKING

Spec, plan, build, review

The single biggest difference between a system that grows and one that collapses is not the AI. It is the order in which you work.

Small tasks you simply ask for. Anything that takes more than an hour (a new panel, a new skill family, a change that touches several parts) goes through the same five steps. The superpowers plugin from Part 2 carries most of this for you.

1 **Brainstorm.** Claude asks you questions, one at a time, until it understands what you actually want. Answer in plain language. This is where most mistakes are prevented.

2 **Spec.** The answers become a short design document: what is being built, what is not, and how you will know it works. You approve it. From then on, the spec decides.

3 **Plan.** The spec becomes a numbered list of small tasks, each with its own check.

4 **Build.** Claude works through the plan, often by handing each task to a separate agent and checking the work. Every task gets a test, so later changes cannot quietly break it.

5 **Review and merge.** A fresh review looks for errors before the work is merged into the main version. You try it yourself before you say yes.

Git is your undo button

Ask Claude to put your system under **git** (version control) from day one, with a private copy on GitHub. Every change is saved as a step you can go back to. New work happens on a separate branch, so the version you use every day keeps working while the new one is being built.

PROMPT · START A LARGER CHANGE

I want to build [feature]. Use the brainstorming skill and ask me one question at a time. When we agree, write a spec, then a plan. Do the work on a separate branch, and do not merge anything until I have tried it and said yes.

PART 4 · WAY OF WORKING

Make every session hand over to the next

Long projects outlive a single conversation. The conversation ends; the files remain. So the files have to carry the thread.

Decision log

Every decision that was not obvious, with the reason. When you wonder in three weeks why something works the way it does, the answer is here, not in your head.

Handover document

At the end of a session: what is done, what is half done, what comes next, and what to read first. The next session starts by reading it.

PROMPT · END OF SESSION

We are stopping here. Write a handover document for the next session: what we finished, what is unfinished, what the next step is, which files to read first, and any lesson from today worth keeping. Update the project's memory file to point to it.

Three rules that protect the system

- **Check costs before running anything paid.** Some scripts call paid services silently. Ask what a command will cost before you approve it, and keep API keys out of your everyday environment.
- **Verify, do not trust.** "Done" means you have seen it work. Ask Claude to show you the evidence: the test that passed, the screenshot, the file.
- **Keep personal data out.** Student information, assessments and colleagues' personal data never go into your AI OS. It is a tool for your own work, not a student register, and your local data protection rules still apply.

JOHAN'S NOTE

The handover document changed how I work more than any feature did. I can close the computer in the middle of a large build, open a new session days later, and be back at full speed within minutes.

YOUR FIRST MONTH

A four-week roadmap

Build in this order. Do not move on to the next week until the current one is in daily use.

Week 1 · Foundation

1. Install Claude Code, create your folder and put it under git with a private GitHub copy.
2. Let Claude interview you and write your `CLAUDE.md`.
3. Set up memory: one file per fact, plus an index.
4. Move your most important reference document (brand, voice or course plan) into the system.

Week 2 · Skills

1. Install the superpowers plugin.
2. Pick the two tasks you repeat most and turn each into a skill.
3. Use them for real, and correct the skill every time it misses.

Week 3 · First panel

1. Build the command centre shell with automatic start.
2. Build the one panel that saves you the most time, and use it daily.

Week 4 · The habit

1. Run your next larger change through brainstorm, spec, plan, build and review.
2. End every session with a handover document.
3. Read your `CLAUDE.md` again and add the rules this month taught you.

> WATCH OUT
>
> The most common way to fail is to build too much too early. A small system you use every day beats a large one you are still building. Let real use decide what the next panel or skill should be.

Build it together with others

This guide is part of the Choosewise community, where educators build and share their AI systems. Bring your questions, your first `CLAUDE.md` and your best skill, and see how others have solved the same problems.

choosewise.education/community

© 2026 Johan Lindström · choosewise.education. For members of the Choosewise community. Not for redistribution.